

Università degli Studi di Roma Tor Vergata

Ingegneria del Software II

Code Smell e Difettosità in Apache Storm

Bug Prediction, What-if Analysis
e Refactoring Automatizzato tramite IA

Report del Progetto

Enrico Barbatano

Luglio 2026

Indice

1	Introduzione	2
2	Metodologia	3
2.1	Milestone 1 – Costruzione del dataset	3
2.1.1	Selezione delle release e raccolta dei ticket	3
2.1.2	Controlli di consistenza sui ticket	3
2.1.3	Determinazione della IV e labeling	3
2.1.4	Estrazione delle metriche e dei code smell	4
2.2	Milestone 2 – Identificazione di modelli accurati	4
2.2.1	Classificatori e preprocessing	4
2.2.2	Feature Selection	5
2.2.3	Bilanciamento e prevenzione del data leakage	5
2.2.4	Validazione e metriche di valutazione	5
2.3	Milestone 3 – Analisi dell'impatto dei code smell	6
2.3.1	Scelta dei BClassifier	6
2.3.2	Costruzione dei dataset	7
2.3.3	Procedura e metriche di valutazione	7
2.3.4	Analisi di correlazione e potere informativo	7
2.4	Milestone 4 – Refactoring automatizzato tramite IA	8
2.4.1	Obiettivo e domande di ricerca	8
2.4.2	Selezione dei SUT	8
2.4.3	Validazione e analisi delle feature	8
3	Risultati	9
3.1	Risultati della Milestone 1 – Costruzione del dataset	9
3.2	Risultati della Milestone 2 – Identificazione di modelli accurati	9
3.2.1	Confronto delle prestazioni	10
3.2.2	Effetto della Feature Selection e di SMOTE	10
3.2.3	Elezione del BClassifier	10
3.3	Risultati della Milestone 3 – Analisi dell'impatto dei code smell	10
3.3.1	RandomForest senza Feature Selection	11
3.3.2	RandomForest con Feature Selection	11
3.3.3	Interpretazione del contrasto	11
3.3.4	Risposta alla domanda di ricerca	12
3.4	Risultati della Milestone 4 – Valutazione del refactoring automatizzato	12
3.4.1	RQ1 – Compilabilità e stabilità	12
3.4.2	RQ2 – Riduzione del debito tecnico	12

3.4.3	RQ3 – Impatto del contesto	12
3.4.4	RQ4 – Metriche e bugginess stimata	12
4	Minacce alla validità e scelte soggettive	13
4.1	Milestone 1 – Costruzione del dataset	13
4.2	Milestone 2 – Identificazione di modelli accurati	13
4.3	Milestone 3 – What-if Analysis	14
4.4	Milestone 4 – Refactoring automatizzato	14
5	Conclusioni	14
5.0.1	Milestone 1 – Costruzione del dataset	14
5.0.2	Milestone 2 – Identificazione del BClassifier	14
5.0.3	Milestone 3 – Impatto dei code smell	14
5.0.4	Milestone 4 – Refactoring automatizzato	14
A	Tabelle metodologiche della costruzione del dataset	15
B	Metriche del dataset	16
C	Configurazioni dei classificatori	17
D	Risultati della What-if Analysis	18
E	Composizione e risultati delle varianti di refactoring	20

1 Introduzione

L'obiettivo del presente studio è analizzare la relazione tra code smell e difettosità nel progetto open source Apache Storm. Apache Storm è un sistema distribuito per l'elaborazione affidabile di flussi di dati in tempo reale. Il codice sorgente è stato estratto dal repository ufficiale del progetto:

<https://github.com/apache/storm.git>

Il lavoro è articolato in quattro milestone:

1. **Milestone 1 – Costruzione del dataset:** raccolta e controllo dei ticket Jira, estrazione classi Java per release, calcolo delle metriche software e assegnazione dell'etichetta buggy a ciascuna coppia classe-release.
2. **Milestone 2 – Identificazione di modelli accurati:** addestramento e valutazione del miglior classificatore, considerando configurazioni differenti.
3. **Milestone 3 – Analisi dell'impatto dei code smell:** analisi controfattuale per stimare quante classi verrebbero predette buggy in uno scenario privo di code smell, mantenendo invariate le altre metriche software.
4. **Milestone 4 – Refactoring automatizzato tramite IA:** valutazione dell'efficacia di Microsoft Copilot nel rifattorizzare classi reali di Apache Storm, riducendo i code smell senza alterare il comportamento osservabile e la qualità del codice.

2 Metodologia

2.1 Milestone 1 – Costruzione del dataset

L'obiettivo della Milestone 1 è costruire un dataset evolutivo nel quale ogni istanza rappresenta una coppia classe-release, descritta mediante metriche strutturali, evolutive, storiche e di debito tecnico e associata a un'etichetta *buggy*.

2.1.1 Selezione delle release e raccolta dei ticket

I dati relativi alle release e ai ticket sono stati recuperati mediante le API REST di Jira. Per mitigare il fenomeno dello *snoring*, cioè il rischio di considerare sane classi contenenti difetti non ancora scoperti, è stato escluso l'ultimo 66% della storia evolutiva. L'analisi è stata quindi condotta sulle prime 12 release valide delle 36 disponibili.

Mediante una query JQL sono stati selezionati esclusivamente i ticket:

- di tipo *Bug*;
- con stato *Closed* oppure *Resolved*;
- con risoluzione *Fixed*.

Sono stati così considerati soltanto difetti confermati, corretti e con un ciclo di vita concluso.

2.1.2 Controlli di consistenza sui ticket

Per garantire l'integrità del dataset, i ticket estratti da Jira sono stati sottoposti a controlli di consistenza funzionali a escludere record incompleti, non collegabili al codice sorgente o caratterizzati da informazioni temporali incoerenti. I ticket non conformi ai criteri riportati nella Tabella 1 sono stati esclusi dal processo di labeling.

2.1.3 Determinazione della IV e labeling

La Injected Version *IV*, cioè la prima release nella quale il difetto è considerato presente, è stata determinata secondo una gerarchia basata sull'affidabilità delle informazioni disponibili.

Quando presenti e coerenti, sono state utilizzate le Affected Versions dichiarate in Jira. In caso contrario, è stato applicato SZZ: a partire dal commit di fix, le righe modificate o rimosse sono state analizzate mediante *git blame* per risalire ai possibili Bug-Introducing Changes.

SZZ non è applicabile quando il fix consiste esclusivamente nell'aggiunta di nuove righe, poiché non esistono righe preesistenti sulle quali eseguire il *blame*. In questi casi, o quando Jira e SZZ non fornivano una *IV* valida, è stata utilizzata la tecnica Proportion Total.

Per ciascun ticket completo, cioè dotato di valori certi per *IV*, *OV* e *FV*, il parametro *P* è stato calcolato come:

$$P_i = \frac{FV_i - IV_i}{FV_i - OV_i}.$$

L'analisi dei ticket completi ha restituito una mediana pari a 2,0 e una media pari a circa 2,76. Per la configurazione finale è stata scelta la media, poiché rappresenta il valore atteso della distribuzione e tiene conto anche dei bug caratterizzati da un ciclo di vita più lungo, evitando di appiattire la stima sui soli casi centrali.

Il valore utilizzato da Proportion Total è stato quindi:

$$\overline{P} = \frac{1}{n} \sum_{i=1}^n P_i = 2,76.$$

Per i ticket incompleti, la Injected Version è stata stimata mediante:

$$\widehat{IV} = FV - (FV - OV) \times \overline{P}.$$

La versione stimata è stata successivamente ricondotta a una release valida e sottoposta ai controlli temporali descritti nella Tabella 1.

Infine, una classe c è stata etichettata come difettosa in una release r secondo la seguente regola:

$$\text{buggy}(c, r) = \begin{cases} \text{YES}, & \text{se } IV \leq r < OV, \\ \text{NO}, & \text{altrimenti.} \end{cases}$$

L'intervallo termina in OV perché il dataset rappresenta il periodo nel quale il difetto era già presente nel codice, ma non era ancora stato segnalato ufficialmente.

2.1.4 Estrazione delle metriche e dei code smell

Per ogni coppia classe-release sono state calcolate oltre venti metriche software, descritte nella Tabella 3, in Appendice B. Le metriche sono state classificate in base al soggetto, distinguendo tra proprietà della classe e informazioni derivate dai commit, e alla modalità di calcolo, separando valori puntuali della classe da valori aggregati sulla storia evolutiva.

Per quantificare il debito tecnico, ogni classe Java presente nelle release selezionate è stata sottoposta ad analisi statica mediante SonarCloud. La feature `code_smells` rappresenta il numero complessivo di violazioni relative a programmazione, manutenibilità e design rilevate sulla versione corrente della classe ed è utilizzata come indicatore del debito tecnico accumulato.

2.2 Milestone 2 – Identificazione di modelli accurati

L'obiettivo della Milestone 2 è individuare la configurazione più accurata per predire la difettosità delle classi, confrontando algoritmi di Machine Learning, selezione delle feature e tecniche di bilanciamento.

2.2.1 Classificatori e preprocessing

La sperimentazione ha confrontato tre classificatori con caratteristiche differenti:

- **RandomForest**, ensemble di alberi decisionali adatto a rappresentare relazioni non lineari e interazioni tra le metriche;
- **NaiveBayes**, classificatore probabilistico basato sull'ipotesi di indipendenza condizionale delle feature;
- **IBk**, classificatore *lazy* basato sull'algoritmo k -Nearest Neighbors e sulla distanza tra le istanze.

Prima dell'addestramento sono stati rimossi gli attributi identificativi `project`, `release` e `nameclass`, affinché i modelli apprendessero dalle proprietà del software e non da nomi o identificativi univoci.

Le feature numeriche sono state inoltre normalizzate per rendere confrontabili metriche caratterizzate da scale differenti. Tale operazione è particolarmente importante per IBk, poiché una feature con valori numericamente più elevati potrebbe dominare il calcolo della distanza anche senza possedere maggiore capacità predittiva.

2.2.2 Feature Selection

È stata valutata una configurazione con Feature Selection di tipo *Filter*, eseguita indipendentemente dal classificatore. In Weka è stato utilizzato l'evaluator `CfsSubsetEval1`, che privilegia sottoinsiemi di feature correlati con la classe target ma poco ridondanti tra loro.

La ricerca del sottoinsieme è stata eseguita mediante `BestFirst` in modalità *forward search*. Il processo parte da un insieme vuoto e aggiunge progressivamente le feature che migliorano maggiormente la qualità del sottoinsieme. Il *backtracking* consente di tornare a configurazioni precedenti quando le aggiunte più recenti non producono ulteriori miglioramenti, riducendo il rischio di arrestare la ricerca in una soluzione locale non ottimale.

Il processo ha individuato le seguenti feature:

```
loc, loc_added, fan_out, revisions_density,  
age, bug_density, code_smells.
```

La feature `code_smells` è stata mantenuta manualmente nel sottoinsieme, poiché necessaria per la What-if Analysis della Milestone 3. Sono state quindi confrontate configurazioni con tutte le metriche e configurazioni basate sul sottoinsieme selezionato.

2.2.3 Bilanciamento e prevenzione del data leakage

Il dataset presenta un forte sbilanciamento: le istanze con `buggy=YES` rappresentano circa il 7,2% del totale. In assenza di bilanciamento, un classificatore potrebbe privilegiare la classe sana, ottenendo un'elevata accuratezza complessiva ma una capacità insufficiente di riconoscere le classi difettose.

Per ridurre questo problema è stata valutata una configurazione con SMOTE. La tecnica genera nuove istanze sintetiche della classe minoritaria interpolando i valori di un'istanza `buggy` con quelli dei suoi vicini appartenenti alla stessa classe. A differenza della semplice duplicazione, SMOTE introduce quindi nuovi esempi artificiali nello spazio delle feature, aumentando la rappresentatività della classe minoritaria.

SMOTE è stato configurato all'interno di un `FilteredClassifier` di Weka, in modo da applicare il bilanciamento esclusivamente ai dati di training di ciascun fold. I dati di test rimangono così reali e non vengono contaminati da istanze sintetiche generate utilizzando informazioni provenienti dal fold di valutazione.

La combinazione dei fattori sperimentali ha prodotto dodici configurazioni:

$$3 \text{ classificatori} \times 2 \text{ configurazioni di Feature Selection} \times 2 \text{ configurazioni di bilanciamento} = 12.$$

2.2.4 Validazione e metriche di valutazione

Le dodici configurazioni sono state valutate mediante una *10-times 10-fold cross-validation*. In ogni ripetizione, il dataset è stato suddiviso in dieci fold: nove sono stati utilizzati per il training e uno per il test, alternando il fold di valutazione fino a coprire l'intero dataset. La procedura è stata ripetuta dieci volte con differenti randomizzazioni.

Questa strategia è stata scelta per ridurre la dipendenza dei risultati da una singola partizione del dataset. Le ripetizioni permettono infatti di osservare il comportamento del modello su diverse composizioni dei fold, fornendo una stima più stabile e meno sensibile a una specifica suddivisione tra training e test.

Considerando `buggy=YES` come classe positiva, Recall e Precision sono definiti come:

$$Recall = \frac{TP}{TP + FN}, \quad Precision = \frac{TP}{TP + FP}.$$

Il Recall misura quante classi realmente buggy vengono individuate, mentre la Precision misura quante delle classi segnalate come buggy lo sono effettivamente. Le due metriche sono complementari perché valutano rispettivamente falsi negativi e falsi positivi.

L'AUC misura la capacità del classificatore di distinguere le classi buggy da quelle sane considerando tutte le possibili soglie decisionali:

$$AUC = \int_0^1 TPR(FPR) d(FPR),$$

dove:

$$TPR = \frac{TP}{TP + FN}, \quad FPR = \frac{FP}{FP + TN}.$$

Un valore elevato di AUC indica che il modello assegna generalmente un rischio maggiore alle classi buggy rispetto a quelle sane.

La Kappa Statistic misura invece l'accordo tra le predizioni e le label reali, correggendolo rispetto all'accordo atteso casualmente:

$$\kappa = \frac{p_o - p_e}{1 - p_e},$$

dove p_o è l'accordo osservato e p_e quello atteso per caso.

La Kappa è stata scelta per compensare l'effetto dello sbilanciamento del dataset, nel quale le istanze buggy rappresentano soltanto il 7,2% del totale. Un classificatore che predicesse sempre la classe sana otterrebbe infatti un'Accuracy prossima al 92,8%, pur senza individuare alcuna classe buggy; la Kappa di tale modello sarebbe invece prossima a zero, evidenziandone la scarsa utilità.

AUC e Kappa valutano quindi aspetti complementari: l'AUC misura la capacità discriminativa e di ranking del modello, mentre la Kappa verifica che l'accordo ottenuto non sia una conseguenza della sola prevalenza della classe sana. Precision e Recall completano l'interpretazione, distinguendo rispettivamente il costo dei falsi positivi e quello dei falsi negativi. Il modello migliore è stato pertanto selezionato privilegiando valori elevati di AUC e Kappa, senza trascurare l'equilibrio tra Precision e Recall.

2.3 Milestone 3 – Analisi dell'impatto dei code smell

L'obiettivo della Milestone 3 è valutare la variazione della difettosità predetta in uno scenario teorico a debito tecnico nullo, rispondendo alla domanda: quante predizioni buggy si sarebbero potute evitare in assenza di code smell?

2.3.1 Scelta dei BClassifier

La What-if Analysis è stata condotta utilizzando due configurazioni di RandomForest, entrambe bilanciate mediante SMOTE:

- una configurazione con il best classifier addestrata sull'intero insieme delle metriche;
- una configurazione con Feature Selection, addestrata sul sottoinsieme individuato mediante `CfsSubsetEval`, nel quale è stata mantenuta la feature `code_smells`.

Il confronto consente di verificare se la risposta all'azzeramento degli smell rimanga stabile oppure dipenda dallo spazio delle feature utilizzato dal modello.

2.3.2 Costruzione dei dataset

A partire dal dataset originale sono stati costruiti quattro insiemi:

- **A**: dataset completo delle coppie classe-release;
- **B+**: sottoinsieme di A contenente le istanze con almeno un code smell;
- **B**: copia controfattuale di B+ nella quale `code_smells` è impostato a zero;
- **C**: sottoinsieme di A contenente le istanze naturalmente prive di smell, utilizzato come controllo.

2.3.3 Procedura e metriche di valutazione

I BClassifier sono stati addestrati sul Dataset A e successivamente utilizzati per produrre le predizioni sui dataset A, B+, B e C. Il confronto principale considera il numero di istanze predette come buggy in B+ e nel corrispondente scenario controfattuale B.

Indicando con $E(D)$ il numero di istanze predette come buggy nel dataset D , la variazione netta è definita come:

$$\Delta Buggy = E(B+) - E(B).$$

Un valore positivo indica una riduzione netta delle predizioni buggy nello scenario senza smell; un valore negativo indica invece un aumento. Il delta rappresenta il saldo tra i conteggi complessivi delle predizioni positive e non necessariamente il numero di specifiche istanze passate da buggy a sane.

La variazione relativa rispetto alle predizioni iniziali in B+ è definita come:

$$Variation_{B+} = \frac{\Delta Buggy}{E(B+)} \times 100.$$

È stata inoltre calcolata la variazione rispetto al numero di predizioni buggy prodotte sull'intero Dataset A:

$$Variation_A = \frac{\Delta Buggy}{E(A)} \times 100.$$

Le due variazioni percentuali sono state calcolate per fornire una lettura relativa del delta. In particolare, $Variation_{B+}$ misura quanto l'azzeramento degli smell incida sulle predizioni relative alle sole classi originariamente smelly, mentre $Variation_A$ rapporta lo stesso cambiamento al numero complessivo di predizioni buggy prodotte sull'intero progetto.

Il Dataset C è stato utilizzato come riferimento per confrontare il comportamento del classificatore sulle classi naturalmente prive di smell con quello osservato sulle classi rese artificialmente prive di smell nel Dataset B.

2.3.4 Analisi di correlazione e potere informativo

Per interpretare i risultati della What-if Analysis sono state utilizzate la correlazione di Pearson e l'Information Gain.

La correlazione di Pearson misura l'intensità del legame lineare tra due feature:

$$r_{X,Y} = \frac{\text{cov}(X, Y)}{\sigma_X \sigma_Y}.$$

La correlazione di Pearson è stata utilizzata per verificare l'eventuale ridondanza tra `code_smells` e le altre metriche. Ad esempio una correlazione elevata con LOC indica che le classi più grandi tendono anche ad avere più smell. Azzerare soltanto gli smell, lasciando invariata la dimensione, può quindi produrre combinazioni

rare nei dati di addestramento e influenzare le predizioni del modello. L'analisi serve pertanto a interpretare eventuali risultati controintuitivi della What-if Analysis.

L'Information Gain misura invece quanto la conoscenza di una feature riduca l'incertezza sulla label buggy:

$$IG(Y, X) = H(Y) - H(Y | X).$$

L'Information Gain consente di confrontare il potere informativo degli smell con quello delle altre metriche. Se altre feature risultano più informative, il modello può continuare a classificare una classe come rischiosa anche dopo l'azzeramento degli smell.

Pearson e Information Gain sono quindi utilizzati per giustificare la risposta dei classificatori: il primo evidenzia eventuali ridondanze tra le feature, mentre il secondo permette di stabilire quanto gli smell contribuiscano alla predizione rispetto agli altri fattori. I valori ottenuti e i risultati della What-if Analysis sono presentati nella sezione dedicata.

2.4 Milestone 4 – Refactoring automatizzato tramite IA

2.4.1 Obiettivo e domande di ricerca

L'obiettivo della Milestone 4 è valutare l'efficacia di Microsoft Copilot nel refactoring automatizzato di classi reali di Apache Storm. In particolare, lo studio verifica se l'LLM riesca a ridurre i code smell senza introdurre regressioni nei comportamenti osservabili e senza peggiorare le metriche software considerate dal modello di Bug Prediction.

La sperimentazione risponde alle seguenti domande:

- **RQ1 – Compilabilità e stabilità:** la variante compila, supera i test disponibili e può sostituire la classe originale?
- **RQ2 – Debito tecnico:** il refactoring elimina gli smell originari senza introdurne di nuovi?
- **RQ3 – Contesto fornito:** l'aggiunta progressiva di test e informazioni dinamiche migliora l'affidabilità del refactoring?
- **RQ4 – Bugginess stimata:** il refactoring modifica le metriche software e la probabilità di difettosità stimata dal classificatore?

2.4.2 Selezione dei SUT

Applicando l'algoritmo di selezione previsto ed escludendo le classi banali, sono stati scelti due casi con caratteristiche opposte:

- **StatsUtil:** God Class utility di 1.709 LOC e 265 code smell, caratterizzata da numerosi metodi statici e strutture dati annidate;
- **RedisKeyValueStateIterator:** classe compatta di circa 70 LOC e un solo code smell, adatta a valutare un intervento localizzato.

La scelta permette di confrontare il comportamento dell'LLM su una classe piccola e focalizzata e su una classe di grandi dimensioni e maggiore complessità.

2.4.3 Validazione e analisi delle feature

Ogni variante è stata validata come sostituta di C_0 . La procedura ha previsto:

1. compilazione della variante nel progetto Apache Storm;

2. esecuzione della suite standard e degli oracoli disponibili;
3. nuova analisi statica dei code smell;
4. ricalcolo delle metriche software;
5. nuova predizione della probabilità di bugginess.

LOC, Fan Out e code smell sono stati calcolati direttamente sul sorgente di ciascuna variante. Le metriche evolutive e storiche sono state invece ricavate mediante `git log -follow` sul percorso del relativo file.

Poiché C_1-C_4 sono state conservate in file distinti, tali metriche descrivono la storia sperimentale delle varianti e non una prosecuzione della storia Git di C_0 . Il confronto è stato pertanto concentrato soprattutto sulle metriche direttamente legate al sorgente corrente, come LOC, Fan Out e `code_smells`.

Per ciascuna feature f , la variazione rispetto alla classe originale è definita come:

$$\Delta f(C_i) = f(C_i) - f(C_0).$$

Le variazioni sono state interpretate considerando l'associazione delle feature con la label `buggy`. Poiché tale associazione non implica causalità, la valutazione è stata completata confrontando la probabilità stimata dal BClassifier:

$$\Delta P_{\text{buggy}}(C_i) = P(\text{buggy}=\text{YES} \mid C_i) - P(\text{buggy}=\text{YES} \mid C_0).$$

Un valore positivo indica un aumento della probabilità stimata, mentre un valore negativo indica una riduzione. La valutazione finale considera quindi compilabilità, superamento dei test, code smell residui, variazione delle metriche direttamente confrontabili e bugginess stimata.

3 Risultati

3.1 Risultati della Milestone 1 – Costruzione del dataset

Il processo di estrazione e filtraggio ha prodotto un dataset evolutivo relativo alle prime 12 release valide di Apache Storm, corrispondenti al primo 33% delle 36 versioni censite su Jira. L'esclusione delle 24 release più recenti ha contribuito a mitigare il fenomeno dello *snoring*, riducendo il rischio di considerare sane classi contenenti difetti non ancora scoperti.

Il dataset finale comprende 9.864 coppie classe-release. Di queste, 713 sono etichettate come `buggy=YES`, pari al 7,23% del totale, mentre le restanti 9.151 appartengono alla classe sana. La distribuzione conferma quindi un marcato sbilanciamento della variabile target.

Il processo di linkage Jira-Git ha collegato correttamente 1.132 dei 1.193 ticket validi estratti, raggiungendo una copertura del 94,89%. L'applicazione di SZZ a livello di classe ha consentito di utilizzare 711 ticket per ricostruire l'introduzione dei difetti e associare i ticket alle classi Java coinvolte.

La caratterizzazione del debito tecnico mostra inoltre che 597 delle 713 classi `buggy` presentano almeno un code smell, corrispondenti all'83,73%. Tale prevalenza motiva l'analisi degli smell come possibili indicatori di rischio, ma non dimostra da sola una relazione causale con la difettosità.

Il riepilogo quantitativo della Milestone 1 è riportato nella Tabella 2, collocata in Appendice A.

3.2 Risultati della Milestone 2 – Identificazione di modelli accurati

La Milestone 2 ha confrontato dodici configurazioni, ottenute combinando tre classificatori, l'impiego o meno della Feature Selection e l'applicazione o meno di SMOTE. I risultati completi sono riportati nella Tabella 4, in Appendice C.

3.2.1 Confronto delle prestazioni

RandomForest ha ottenuto le prestazioni complessivamente migliori. NaiveBayes ha raggiunto valori elevati di Recall, ma associati a Precision e Kappa molto basse, indicando un numero elevato di falsi positivi. IBk ha mostrato un comportamento più equilibrato, ma valori di AUC e Kappa inferiori rispetto a RandomForest.

Tra le configurazioni RandomForest, quella senza Feature Selection e senza SMOTE ha ottenuto la maggiore AUC, pari a 0,962, e la maggiore Precision, pari a 0,857. La configurazione senza Feature Selection e con SMOTE ha invece raggiunto il Recall più elevato, pari a 0,577, e la maggiore Kappa, pari a 0,649, mantenendo AUC e Precision rispettivamente pari a 0,958 e 0,806.

Il confronto con ZeroR evidenzia inoltre i limiti della sola Accuracy. Poiché il 92,8% delle coppie classe-release appartiene alla classe sana, un classificatore che predicesse sempre `buggy=NO` otterrebbe un'Accuracy del 92,8%, pur senza individuare alcuna coppia classe-release realmente buggy e con Recall e Kappa pari a zero.

3.2.2 Effetto della Feature Selection e di SMOTE

Per RandomForest con SMOTE, la riduzione da 21 a 7 feature ha determinato una diminuzione di tutte le metriche considerate:

$$\begin{aligned}AUC &: 0,958 \rightarrow 0,942, \\ \kappa &: 0,649 \rightarrow 0,595, \\ Recall &: 0,577 \rightarrow 0,530, \\ Precision &: 0,806 \rightarrow 0,749.\end{aligned}$$

Nelle condizioni sperimentali adottate, l'insieme completo delle metriche consente quindi a RandomForest di sfruttare meglio le relazioni strutturali, storiche ed evolutive presenti nel dataset. La configurazione con Feature Selection è stata comunque mantenuta nella Milestone 3 per verificare se la risposta all'azzeramento dei code smell dipendesse dall'insieme di metriche utilizzato dal classificatore.

Anche l'effetto di SMOTE varia in base al classificatore. Nel caso di RandomForest senza Feature Selection, il bilanciamento aumenta il Recall da 0,523 a 0,577 e la Kappa da 0,627 a 0,649, riducendo moderatamente la Precision da 0,857 a 0,806 e l'AUC da 0,962 a 0,958.

SMOTE ha quindi permesso a RandomForest di riconoscere una quota maggiore della classe minoritaria, accettando un moderato aumento dei falsi positivi. Il bilanciamento non ha però prodotto lo stesso beneficio per tutti i classificatori e la sua efficacia è stata valutata insieme alle caratteristiche del modello.

3.2.3 Elezione del BClassifier

Sulla base dei confronti precedenti, RandomForest senza Feature Selection e con SMOTE è stato selezionato come BClassifier principale. Pur presentando AUC e Precision lievemente inferiori rispetto alla configurazione non bilanciata, raggiunge i migliori valori di Recall e Kappa tra le RandomForest.

La configurazione offre quindi il compromesso più adeguato tra capacità discriminativa, affidabilità delle segnalazioni e riconoscimento della classe minoritaria. Tale criterio è particolarmente rilevante nella defect prediction, poiché un falso negativo può escludere un caso realmente buggy dalle successive attività di ispezione.

3.3 Risultati della Milestone 3 – Analisi dell'impatto dei code smell

La What-if Analysis ha confrontato le predizioni sul Dataset B+, contenente le classi con code smell, con quelle sul Dataset B, ottenuto azzerando la sola feature `code_smells`. Le altre metriche sono rimaste invariate. L'analisi è stata eseguita con le due configurazioni RandomForest selezionate nella Milestone 2.

Il confronto quantitativo completo è riportato nella Tabella 5, in Appendice D.

3.3.1 RandomForest senza Feature Selection

La configurazione basata sulle 21 feature ha prodotto 547 predizioni buggy nel Dataset B+ e 525 nel Dataset B:

$$\Delta Buggy = E(B+) - E(B) = 547 - 525 = 22.$$

Nello scenario senza smell si osserva quindi una riduzione netta di 22 predizioni buggy, pari al 4,02% delle predizioni positive in B+ e al 3,11% di quelle prodotte sull'intero Dataset A.

Il beneficio è presente ma contenuto. RandomForest combina le decisioni di numerosi alberi costruiti utilizzando campioni e sottoinsiemi differenti delle feature. Di conseguenza, azzerare `code_smells` modifica soltanto una parte delle decisioni: gli alberi basati sulle altre metriche possono continuare ad assegnare un rischio elevato alle stesse classi.

3.3.2 RandomForest con Feature Selection

La configurazione basata sulle sette feature selezionate ha prodotto 498 predizioni buggy nel Dataset B+ e 539 nel Dataset B:

$$\Delta Buggy = E(B+) - E(B) = 498 - 539 = -41.$$

Il valore negativo indica un aumento netto di 41 predizioni buggy, pari all'8,23% rispetto alle predizioni iniziali in B+. Tale incremento è associato soprattutto all'aumento dei falsi positivi, passati da 62 a 296.

Questo risultato non indica che la rimozione degli smell introduca nuovi difetti. Mostra, invece, che il classificatore con Feature Selection reagisce in modo differente alla combinazione controfattuale delle metriche.

3.3.3 Interpretazione del contrasto

La differenza tra le due configurazioni è riconducibile alle relazioni apprese da RandomForest. Come riportato nella Tabella 6, LOC e `code_smells` presentano una correlazione quasi perfetta:

$$r = 0,988595.$$

Nelle istanze utilizzate per l'addestramento, valori elevati di LOC sono quindi generalmente associati a un numero elevato di smell. Nel Dataset B viene però azzerato soltanto `code_smells`, lasciando invariata LOC. Si producono così combinazioni poco rappresentate nei dati originali, caratterizzate da molte LOC e nessuno smell.

La configurazione con 21 feature può distribuire la decisione su un numero maggiore di segnali ed è quindi meno sensibile a questa combinazione. La configurazione con Feature Selection utilizza soltanto sette metriche, tra cui LOC e `code_smells`: la rottura della relazione tra queste due feature incide maggiormente sui percorsi decisionali degli alberi e contribuisce all'aumento dei falsi positivi.

L'Information Gain, riportato nella Tabella 7, mostra inoltre che LOC, Fan Out e Age sono più informative degli smell rispetto alla label buggy. In particolare:

$$IG(LOC) = 0,042880, \quad IG(\text{code_smells}) = 0,017780.$$

LOC risulta quindi circa 2,4 volte più informativa degli smell. Poiché LOC, Fan Out e Age rimangono invariate nel Dataset B, il classificatore continua a ricevere segnali di rischio anche dopo l'azzeramento degli smell.

Questo spiega perché la configurazione senza Feature Selection produca soltanto una riduzione contenuta e perché quella con Feature Selection possa generare un risultato opposto.

3.3.4 Risposta alla domanda di ricerca

Il BClassifier principale stima una riduzione netta di **22 predizioni buggy**, pari al **4,02%** delle predizioni positive in B+.

La configurazione con Feature Selection produce invece 41 predizioni buggy aggiuntive. La stima non risulta quindi stabile rispetto all'insieme di metriche utilizzato dal modello.

Il valore 22 rappresenta la risposta del BClassifier principale all'azzeramento simulato degli smell e non dimostra che 22 difetti reali sarebbero stati certamente prevenuti.

3.4 Risultati della Milestone 4 – Valutazione del refactoring automatizzato

I risultati della Milestone 4 sono organizzati in base alle quattro domande di ricerca. Le tabelle complete sono riportate in Appendice E.

3.4.1 RQ1 – Compilabilità e stabilità

Tutte le varianti di entrambi i SUT compilano e superano la suite di test. Le varianti mantengono package e API pubbliche e possono quindi essere considerate *drop-in replacement* rispetto ai comportamenti verificati dai test disponibili.

Il risultato non dimostra un'equivalenza semantica assoluta, ma fornisce evidenza dell'assenza di regressioni nei comportamenti coperti dagli oracoli utilizzati. Gli esiti della validazione sono riportati nella Tabella 9.

3.4.2 RQ2 – Riduzione del debito tecnico

Per `RedisKeyValueStateIterator`, l'unico smell presente in C_0 è stato eliminato in tutte le varianti.

Per `StatsUtil`, il numero di smell è passato da 265 in C_0 a 19 in C_1 , 45 in C_2 e 19 in C_3 e C_4 . La riduzione massima è quindi pari a 246 smell. I risultati completi sono riportati nella Tabella 10.

3.4.3 RQ3 – Impatto del contesto

Per `RedisKeyValueStateIterator`, le quattro varianti producono lo stesso risultato statico e funzionale. L'aggiunta progressiva di informazioni non determina quindi differenze osservabili dopo che l'unico smell è stato eliminato in C_1 .

Per `StatsUtil`, il risultato non mostra un miglioramento monotono all'aumentare del contesto: C_2 conserva 45 smell, mentre C_1 , C_3 e C_4 ne conservano 19. Tutte le varianti compilano e superano la suite standard. Non è pertanto possibile concludere che una maggiore quantità di contesto produca automaticamente un refactoring migliore secondo tutte le metriche.

3.4.4 RQ4 – Metriche e bugginess stimata

Tutte le varianti mantengono la classificazione nominale `buggy=NO`, ma presentano un lieve aumento della probabilità stimata rispetto a C_0 .

Per `RedisKeyValueStateIterator`, LOC rimane pari a 69, i code smell passano da 1 a 0 e il Fan Out stimato aumenta da 44 a 45. La probabilità di bugginess passa dal 34% al 36%.

Per `StatsUtil`, C_2 conserva 45 code smell e presenta un Fan Out pari a 153, ottenendo una probabilità del 35%. C_1 , C_3 e C_4 riducono invece gli smell a 19, ma raggiungono un Fan Out pari a 155 e una probabilità del 38%.

Il risultato mostra che il numero di code smell non determina da solo la probabilità stimata. Come riportato nella Tabella 7, il Fan Out presenta un Information Gain pari a 0,036970, superiore a quello dei code smell, pari a 0,017780. Il minore Fan Out di C_2 può quindi contribuire a spiegare perché questa variante presenti una probabilità inferiore, nonostante il maggior numero di smell residui.

L'Information Gain non dimostra tuttavia un rapporto causale sulla singola predizione. RandomForest considera congiuntamente tutte le feature e le relative interazioni; il risultato indica pertanto che il beneficio derivante dalla riduzione degli smell può essere compensato dal peggioramento di altre metriche strutturali.

Le Tabelle 11 e 13 sintetizzano LOC, Fan Out, code smell e probabilità stimata. Le metriche complete sono riportate nelle Tabelle 12 e 14.

4 Minacce alla validità e scelte soggettive

Questa sezione presenta le principali limitazioni della sperimentazione e le scelte metodologiche che possono influenzare l'interpretazione e la generalizzabilità dei risultati.

4.1 Milestone 1 – Costruzione del dataset

Mapping tra release, commit e branch. La presenza di più branch di sviluppo rende non sempre univoca l'associazione tra commit e release. Ciò può introdurre imprecisioni nelle metriche temporali, come Age, e nella ricostruzione della storia delle classi. L'ispezione effettuata indica che il problema interessa meno del 4% delle coppie classe-release.

SZZ e Proportion. SZZ dipende dalla corretta identificazione del commit di fix e non è applicabile quando la correzione consiste soltanto nell'aggiunta di nuove righe. Nei casi incompleti è stata quindi utilizzata Proportion, che introduce inevitabilmente un margine di errore nella stima della Injected Version.

Scelta del parametro P . Per Proportion è stata utilizzata la media, pari a 2,76, anziché la mediana, pari a 2,0. La scelta consente di considerare anche i bug con cicli di vita più lunghi, ma rende la stima maggiormente sensibile agli outlier.

Selezione della Fixed Version. In presenza di più Fixed Versions è stata selezionata la versione cronologicamente più vecchia, assumendo che rappresenti il primo rilascio contenente la correzione. La scelta può risultare imprecisa se la prima versione contiene soltanto una correzione parziale.

4.2 Milestone 2 – Identificazione di modelli accurati

Temporal leakage. La *10-times 10-fold cross-validation* assegna casualmente le istanze ai fold senza rispettare l'ordine temporale delle release. Snapshot della stessa classe appartenenti a release differenti possono quindi comparire nei dati di training e di test, producendo stime potenzialmente ottimistiche.

Configurazione di SMOTE. SMOTE è stato configurato al 100%, raddoppiando le istanze della classe minoritaria nei dati di training. La percentuale rappresenta un compromesso tra una maggiore rappresentazione degli snapshot buggy e il rischio di modificare eccessivamente la distribuzione originale mediante dati sintetici.

4.3 Milestone 3 – What-if Analysis

Scenario controfattuale semplificato. Nel Dataset B è stata azzerata soltanto la feature `code_smells`, lasciando invariate tutte le altre metriche. Un refactoring reale potrebbe invece modificare contemporaneamente LOC, Fan Out, Churn e altre caratteristiche. La What-if Analysis misura pertanto la risposta del classificatore a uno scenario artificiale e non il numero di difetti realmente prevenuti.

4.4 Milestone 4 – Refactoring automatizzato

Non determinismo del modello generativo. Microsoft Copilot può produrre refactoring differenti a partire dallo stesso prompt. I risultati dipendono quindi dalla specifica esecuzione, dalla formulazione dei prompt e dalle informazioni fornite al modello e non sono deterministicamente riproducibili.

Selezione dei SUT. Sono state escluse dalla selezione preliminare le classi che presentavano congiuntamente meno di 70 LOC e meno di 5 metodi, così da evitare casi di refactoring banali. La valutazione finale riguarda tuttavia soltanto due classi di Apache Storm e non è automaticamente generalizzabile all'intero progetto o ad altri sistemi software.

5 Conclusioni

5.0.1 Milestone 1 – Costruzione del dataset

È stato prodotto un dataset di 9.864 coppie classe-release, di cui 713 etichettate come buggy. Il forte sbilanciamento verso la classe sana ha richiesto specifiche tecniche di bilanciamento durante l'addestramento dei classificatori.

5.0.2 Milestone 2 – Identificazione del BClassifier

RandomForest senza Feature Selection e con SMOTE è risultato il miglior compromesso tra le dodici configurazioni analizzate, con AUC pari a 0,958 e Kappa pari a 0,649. La configurazione bilanciata ha migliorato il riconoscimento della classe buggy, mantenendo elevate capacità discriminativa e Precision.

5.0.3 Milestone 3 – Impatto dei code smell

Il BClassifier principale ha stimato 22 predizioni buggy nette in meno nello scenario senza smell, pari al 4,02% delle predizioni relative agli snapshot smelly. Il risultato non è però stabile con Feature Selection. La causa principale è che LOC, Fan Out e Age risultano più informative degli smell e rimangono invariate nello scenario controfattuale.

5.0.4 Milestone 4 – Refactoring automatizzato

Tutte le varianti prodotte da Microsoft Copilot compilano e superano i test disponibili. Gli smell sono stati eliminati in `RedisKeyValueStateIterator` e ridotti da 265 a valori compresi tra 19 e 45 in `StatsUtil`. La probabilità stimata di difettosità è tuttavia aumentata lievemente, soprattutto per l'incremento del Fan Out.

Nel complesso, la rimozione dei code smell migliora la qualità statica, ma non garantisce da sola una riduzione della difettosità. Il refactoring tramite IA costituisce un supporto efficace, purché sia accompagnato da testing, analisi delle metriche e supervisione umana.

A Tabelle metodologiche della costruzione del dataset

Tabella 1: Controlli di consistenza applicati ai ticket di Apache Storm

Soggetto	Controllo di consistenza	Motivazione
Ticket	Tipo <i>Bug</i> e risoluzione <i>Fixed</i>	Il controllo concentra l'analisi sui difetti reali e ufficialmente confermati come corretti.
Ticket	Stato <i>Closed</i> oppure <i>Resolved</i>	Garantisce che il ciclo di vita del bug sia concluso e che la Fixed Version rappresenti una versione di correzione definitiva.
Ticket	Presenza del linkage con il repository Git	Il collegamento è necessario per individuare il commit di fix, identificare le classi coinvolte e applicare l'algoritmo SZZ.
Ticket	Vincolo temporale $FV \geq OV$	Un bug non può risultare ufficialmente corretto prima della sua apertura.
Ticket	Presenza di almeno un file .java di produzione	Vengono considerate soltanto le classi Java del codice di produzione, escludendo test, documentazione e file non pertinenti.
Ticket dopo Proportion	Vincolo temporale $IV < OV$	La versione stimata di introduzione deve precedere logicamente la versione di apertura del ticket.

Tabella 2: Riepilogo del dataset prodotto per Apache Storm

Proprietà	Valore
Progetto	Apache Storm
Versioni Jira censite	36
Release considerate	12, corrispondenti al primo 33%
Release escluse per mitigare lo <i>snoring</i>	24
Classi totali	9.864
Classi etichettati come buggy=YES	713, pari al 7,23%
Classi etichettati come buggy=NO	9.151, pari al 92,77%
Ticket Jira validi estratti	1.193
Ticket collegati correttamente ai commit Git	1.132, pari al 94,89%
Ticket utilizzati per il labeling mediante SZZ	711
Classi buggy con almeno un code smell	597, pari all'83,73% delle classi buggy

B Metriche del dataset

La Tabella 3 riporta le feature calcolate per ciascuna coppia classe-release. La classificazione distingue le metriche puntuali, riferite allo snapshot della release, dalle metriche aggregate, ricavate dalla storia evolutiva della classe.

Tabella 3: Metriche software calcolate per ogni coppia classe-release

Metrica	Tipo	Descrizione
LOC	Classe – puntuale	Numero complessivo di righe di codice non vuote presenti nello snapshot della classe.
code_smells	Classe – puntuale	Numero di violazioni di manutenibilità rilevate mediante analisi statica sulla versione della classe presente nella release corrente.
Age	Classe – puntuale	Età della classe, espressa in settimane, calcolata a partire dal primo commit che la contiene.
Weighted_Age	Classe – puntuale	Età della classe pesata rispetto alla sua dimensione, utilizzata come indicatore del costo di manutenzione accumulato.
Churn	Classe – aggregata	Somma delle linee aggiunte, eliminate e modificate nelle revisioni dell'intervallo considerato.
LOC_Added	Classe – aggregata	Somma delle linee di codice aggiunte nelle revisioni della classe.
Max_LOC_Added	Classe – aggregata	Numero massimo di linee aggiunte alla classe in una singola revisione.
Avg_LOC_Added	Classe – aggregata	Media delle linee aggiunte nelle revisioni che hanno interessato la classe.
Max_Churn	Classe – aggregata	Valore massimo di Churn registrato in una singola revisione.
NR	Classe – aggregata	Numero di revisioni che hanno interessato la classe nell'intervallo considerato.
NFix	Classe – aggregata	Numero di interventi di bug fixing che hanno coinvolto la classe nell'intervallo considerato.
NAuth	Classe – aggregata	Numero di autori distinti che hanno modificato la classe nell'intervallo considerato.
NDEV	Commit – aggregata	Numero di sviluppatori coinvolti nei commit considerati per la classe.
Change_Set_Size	Commit – aggregata	Numero di file modificati insieme alla classe, utilizzato come indicatore dell'ampiezza del cambiamento.
Entropy	Commit – aggregata	Misura della distribuzione delle modifiche tra i file coinvolti nei commit associati alla classe.
NS	Commit – aggregata	Numero di sottosistemi modificati nei commit che coinvolgono la classe.
FIX	Commit – puntuale	Indicatore booleano che segnala se la classe è stata coinvolta in un'attività di bug fixing.
NFix_Total	Classe – aggregata	Numero complessivo di bug fix che hanno interessato la classe nella sua intera storia evolutiva.
NR_Total	Classe – aggregata	Numero complessivo di revisioni subite dalla classe dalla sua introduzione nel progetto.
Avg_Churn_Total	Classe – aggregata	Valore medio del Churn calcolato sull'intera storia evolutiva della classe.
NAuth_Total	Classe – aggregata	Numero complessivo di autori distinti che hanno modificato la classe dalla sua introduzione.

C Configurazioni dei classificatori

La Tabella 4 riporta le prestazioni delle dodici configurazioni ottenute combinando classificatore, Feature Selection e bilanciamento SMOTE. Le metriche sono state calcolate mediante *10-times 10-fold cross-validation* e fanno riferimento alla classe positiva buggy=YES.

Tabella 4: Metriche delle dodici configurazioni analizzate

#	Feature Selection	Classificatore	SMOTE	Recall	Precision	AUC	Kappa
1	Sì	IBk	No	0,299	0,314	0,681	0,253
2	Sì	IBk	Sì	0,348	0,306	0,690	0,269
3	Sì	NaiveBayes	No	0,881	0,098	0,713	0,054
4	Sì	NaiveBayes	Sì	0,904	0,093	0,711	0,043
5	Sì	RandomForest	No	0,486	0,831	0,948	0,590
6	Sì	RandomForest	Sì	0,530	0,749	0,942	0,595
7	No	IBk	No	0,343	0,355	0,668	0,298
8	No	IBk	Sì	0,381	0,331	0,680	0,299
9	No	NaiveBayes	No	0,953	0,083	0,743	0,023
10	No	NaiveBayes	Sì	0,956	0,082	0,746	0,020
11	No	RandomForest	No	0,523	0,857	0,962	0,627
12	No	RandomForest	Sì	0,577	0,806	0,958	0,649

D Risultati della What-if Analysis

Tabella 5: Confronto dei risultati della What-if Analysis

Configurazione	Buggy B+	Buggy B	$\Delta Buggy$	Variazione su B+
RandomForest senza Feature Selection	547	525	+22	Riduzione netta del 4,02%
RandomForest con Feature Selection	498	539	-41	Aumento netto dell'8,23%

Tabella 6: Correlazione di Pearson delle feature con code_smells

Pos.	Feature	Correlazione code_smells	Interpretazione
1	loc	0,988595	Positiva molto forte, quasi perfetta.
2	avg_churn	0,464234	Positiva moderata.
3	avg_loc_added	0,456895	Positiva moderata.
4	max_churn	0,436330	Positiva moderata.
5	max_loc_added	0,394287	Positiva moderata.
6	loc_touched_total	0,356275	Positiva moderata.
7	churn	0,330372	Positiva moderata.
8	loc_added	0,317227	Positiva moderata.
9	weighted_age	0,314170	Positiva moderata.
10	fan_out	0,084530	Positiva molto debole.
11	ndev	0,035022	Positiva trascurabile.
12	nauth	0,035022	Positiva trascurabile.
13	revisions	0,021807	Positiva trascurabile.
14	fan_in_total	0,008933	Positiva trascurabile.
15	nfix	0,007511	Positiva trascurabile.
16	buggy	0,003201	Relazione lineare praticamente nulla.
17	ns	0,000000	Nessuna associazione lineare.
18	age	-0,000936	Negativa trascurabile.
19	fix	-0,005207	Negativa trascurabile.
20	bug_density	-0,017549	Negativa trascurabile.
21	revisions_density	-0,055952	Negativa molto debole.

Tabella 7: Information Gain delle feature rispetto alla label buggy

Posizione	Feature	Information Gain
1	loc	0,042880
2	fan_out	0,036970
3	age	0,029140
4	code_smells	0,017780
5	revisions_density	0,015730
6	bug_density	0,013200
7	ndev	0,010910
8	nauth	0,010910
9	nfix	0,010500
10	revisions	0,009560
11	fix	0,006980

Continua nella pagina successiva

Posizione	Feature	Information Gain
12	fan_in_total	0,006320
13	avg_loc_added	0,004810
14	avg_churn	0,004710
15	loc_added	0,004680
16	churn	0,004360
17	max_churn	0,004260
18	max_loc_added	0,004110
19	loc_touched_total	0,004070
20	weighted_age	0,003820
21	ns	0,000000

E Composizione e risultati delle varianti di refactoring

La Tabella 8 sintetizza il contesto fornito a Microsoft Copilot per generare le quattro varianti rifattorizzate a partire dalla classe originale.

Le Tabelle 9, 10 e ?? riportano rispettivamente gli esiti della validazione, la variazione dei code smell e la probabilità di difettosità stimata.

Le Tabelle 12 e 14 presentano infine le metriche complete e i relativi delta rispetto a C_0 .

Tabella 8: Composizione delle varianti di refactoring

Variante	Livello di contesto	Informazioni fornite a Microsoft Copilot
C_0	Classe originale	Sorgente Java originale utilizzato come baseline del confronto.
C_1	Contesto statico	Sorgente originale e report dei code smell rilevati mediante analisi statica.
C_2	Specifica funzionale	Contesto di C_1 e suite black-box progettata mediante Category Partition.
C_3	Analisi strutturale	Contesto di C_2 , test Control-Flow e report di Instruction e Branch Coverage prodotti da JaCoCo.
C_4	Rafforzamento degli oracoli	Contesto di C_3 , report PIT e test progettati sulla base dei mutanti sopravvissuti.

Tabella 9: Validazione funzionale delle varianti rifattorizzate

Classe	Variante	Compilazione	Suite standard	Esito
RedisKeyValueStateIterator	C_1	Superata	Superata	Validata come sostituta di C_0 rispetto ai test disponibili.
RedisKeyValueStateIterator	C_2	Superata	Superata	Validata come sostituta di C_0 rispetto ai test disponibili.
RedisKeyValueStateIterator	C_3	Superata	Superata	Validata come sostituta di C_0 rispetto ai test disponibili.
RedisKeyValueStateIterator	C_4	Superata	Superata	Validata come sostituta di C_0 rispetto ai test disponibili.
StatsUtil	C_1	Superata	Superata	Validata come sostituta di C_0 rispetto ai test disponibili.
StatsUtil	C_2	Superata	Superata	Validata come sostituta di C_0 rispetto ai test disponibili.
StatsUtil	C_3	Superata	Superata	Validata come sostituta di C_0 rispetto ai test disponibili.
StatsUtil	C_4	Superata	Superata	Validata come sostituta di C_0 rispetto ai test disponibili.

Tabella 10: Code smell rilevati sulle varianti rifattorizzate

Classe	C_0	C_1	C_2	C_3	C_4	Riduzione massima
RedisKeyValueStateIterator	1	0	0	0	0	-1
StatsUtil	265	19	45	19	19	-246

La riduzione massima corrisponde alla differenza tra il minor numero di smell ottenuto dalle varianti e il valore della classe originale C_0 . Per *StatsUtil*, la riduzione da 265 a 19 smell corrisponde al 92,83%.

Tabella 11: Metriche ricalcolate e bugginess stimata per RedisKeyValueStateIterator

Variante	LOC (Δ)	Fan Out (Δ)	Code Smell (Δ)	Predizione	$P(\text{buggy}=\text{YES})$	ΔP
C_0	69	44	1	NO	34%	-
C_1	69 (0)	45 (+1)	0 (-1)	NO	36%	+2 p.p.
C_2	69 (0)	45 (+1)	0 (-1)	NO	36%	+2 p.p.
C_3	69 (0)	45 (+1)	0 (-1)	NO	36%	+2 p.p.
C_4	69 (0)	45 (+1)	0 (-1)	NO	36%	+2 p.p.

Tabella 12: Metriche complete di RedisKeyValueStateIterator e delta rispetto a C_0

Metrica	C_0	C_1 (Δ)	C_2 (Δ)	C_3 (Δ)	C_4 (Δ)
loc	69	69 (0)	69 (0)	69 (0)	69 (0)
churn	109	218 (+109)	218 (+109)	218 (+109)	218 (+109)
loc_added	108	109 (+1)	109 (+1)	109 (+1)	109 (+1)
fan_out	44	45 (+1)	45 (+1)	45 (+1)	45 (+1)
revisions	2	5 (+3)	5 (+3)	5 (+3)	5 (+3)
max_churn	107	109 (+2)	109 (+2)	109 (+2)	109 (+2)
max_loc_added	107	109 (+2)	109 (+2)	109 (+2)	109 (+2)
avg_churn	54,5	43,6 (-10, 9)	43,6 (-10, 9)	43,6 (-10, 9)	43,6 (-10, 9)
avg_loc_added	54,0	21,8 (-32, 2)	21,8 (-32, 2)	21,8 (-32, 2)	21,8 (-32, 2)
fan_in_total	14	0 (-14)	0 (-14)	0 (-14)	0 (-14)
loc_touched_total	109	218 (+109)	218 (+109)	218 (+109)	218 (+109)
revisions_density	0,028986	0,072464 (+0, 043478)	0,072464 (+0, 043478)	0,072464 (+0, 043478)	0,072464 (+0, 043478)
ns	10	10 (0)	10 (0)	10 (0)	10 (0)
age	0	33 (+33)	33 (+33)	33 (+33)	33 (+33)
weighted_age	33,9021	1,0346 (-32, 8675)	1,0346 (-32, 8675)	1,0346 (-32, 8675)	1,0346 (-32, 8675)
nauth	1	1 (0)	1 (0)	1 (0)	1 (0)
nfix	0	0 (0)	0 (0)	0 (0)	0 (0)
ndev	1	1 (0)	1 (0)	1 (0)	1 (0)
bug_density	0	0 (0)	0 (0)	0 (0)	0 (0)
fix	0	0 (0)	0 (0)	0 (0)	0 (0)
code_smells	1	0 (-1)	0 (-1)	0 (-1)	0 (-1)
buggy (label)	0	0 (0)	0 (0)	0 (0)	0 (0)

Tabella 13: Metriche ricalcolate e bugginess stimata per StatsUtil

Variante	LOC (Δ)	Fan Out (Δ)	Code Smell (Δ)	Predizione	$P(\text{buggy}=\text{YES})$	ΔP
C_0	1709	152	265	NO	33%	–
C_1	1703 (–6)	155 (+3)	19 (–246)	NO	38%	+5 p.p.
C_2	1709 (0)	153 (+1)	45 (–220)	NO	35%	+2 p.p.
C_3	1703 (–6)	155 (+3)	19 (–246)	NO	38%	+5 p.p.
C_4	1704 (–5)	155 (+3)	19 (–246)	NO	38%	+5 p.p.

Tabella 14: Metriche complete di StatsUtil e delta rispetto a C_0

Metrica	C_0	C_1 (Δ)	C_2 (Δ)	C_3 (Δ)	C_4 (Δ)
loc	1709	1703 (–6)	1709 (0)	1703 (–6)	1704 (–5)
churn	2729	4744 (+2015)	4754 (+2025)	4752 (+2023)	4746 (+2017)
loc_added	2553	2372 (–181)	2377 (–176)	2376 (–177)	2373 (–180)
fan_out	152	155 (+3)	153 (+1)	155 (+3)	155 (+3)
revisions	3	6 (+3)	4 (+1)	6 (+3)	4 (+1)
max_churn	2377	2372 (–5)	2377 (0)	2376 (–1)	2373 (–4)
max_loc_added	2377	2372 (–5)	2377 (0)	2376 (–1)	2373 (–4)
avg_churn	909,6667	790,6667	1188,5	792,0	1186,5
		(–119)	(+278, 8333)	(–117, 6667)	(+276, 8333)
avg_loc_added	851,0	395,3333	594,25	396,0 (–455)	593,25
		(–455, 6667)	(–256, 75)		(–257, 75)
fan_in_total	17	0 (–17)	0 (–17)	0 (–17)	0 (–17)
loc_touched_total	2729	4744 (+2015)	4754 (+2025)	4752 (+2023)	4746 (+2017)
revisions_density	0,001755	0,003523	0,002341	0,003523	0,002347
		(+0, 001768)	(+0, 000586)	(+0, 001768)	(+0, 000592)
ns	8	8 (0)	8 (0)	8 (0)	8 (0)
age	33	33 (0)	33 (0)	33 (0)	33 (0)
weighted_age	29,6124	0,6171	0,6047	0,6171	0,6047
		(–28, 9953)	(–29, 0077)	(–28, 9953)	(–29, 0077)
nauth	1	1 (0)	1 (0)	1 (0)	1 (0)
nfix	0	0 (0)	0 (0)	0 (0)	0 (0)
ndev	1	1 (0)	1 (0)	1 (0)	1 (0)
bug_density	0	0 (0)	0 (0)	0 (0)	0 (0)
fix	0	0 (0)	0 (0)	0 (0)	0 (0)
code_smells	265	19 (–246)	45 (–220)	19 (–246)	19 (–246)
buggy (label)	0	0 (0)	0 (0)	0 (0)	0 (0)